CS-Study 3회차 · Network

TCP vs UDP, 핸드셰이크와 제어 기법, 그리고 포트 포워딩

SYN SYN+ACK ACK

발표 · 권혜강

2026. 11. 10

AGENDA

오늘 다룰 내용

01	TCP vs UDP	신뢰성이냐 속도냐, 그리고 UDP가 여전히 쓰이는 이유
02	TCP Handshake	연결은 3번, 종료는 4번 주고받는 이유와 TIME_WAIT
03	TCP 제어 기법	흐름 제어 · 혼잡 제어 · 오류 제어
04	포트 포워딩	사설 IP, NAPT, 공유기의 DNAT 규칙
05	생각해볼 질문	ngrok은 어떻게 동작할까? 포트가 고갈되면?

2회차 흐름 DNS → TCP 연결 → TLS → HTTP 중에서, 오늘은 'TCP 연결'을 깊게 봅니다.

01 · TCP vs UDP

IP는 건물 주소, 포트는 호수

인터넷 계층

IP

‘어느 컴퓨터로?’를 정함. 최선을 다할 뿐(best-effort), 유실·순서 뒤바뀜은 책임지지 않음

전송 계층

포트

‘그 컴퓨터의 어느 프로그램으로?’를 정함. 16비트 번호 0~65535

전송 계층

TCP / UDP

IP 위에서 ‘어떻게’ 보낼지 선택. 신뢰성을 더할지(TCP), 그대로 빠르게 보낼지(UDP)

`22 SSH` `53 DNS` `80 HTTP` `443 HTTPS` `3306 MySQL` `8000 Django runserver`

주황 = 주로 TCP, 파랑 = 주로 UDP

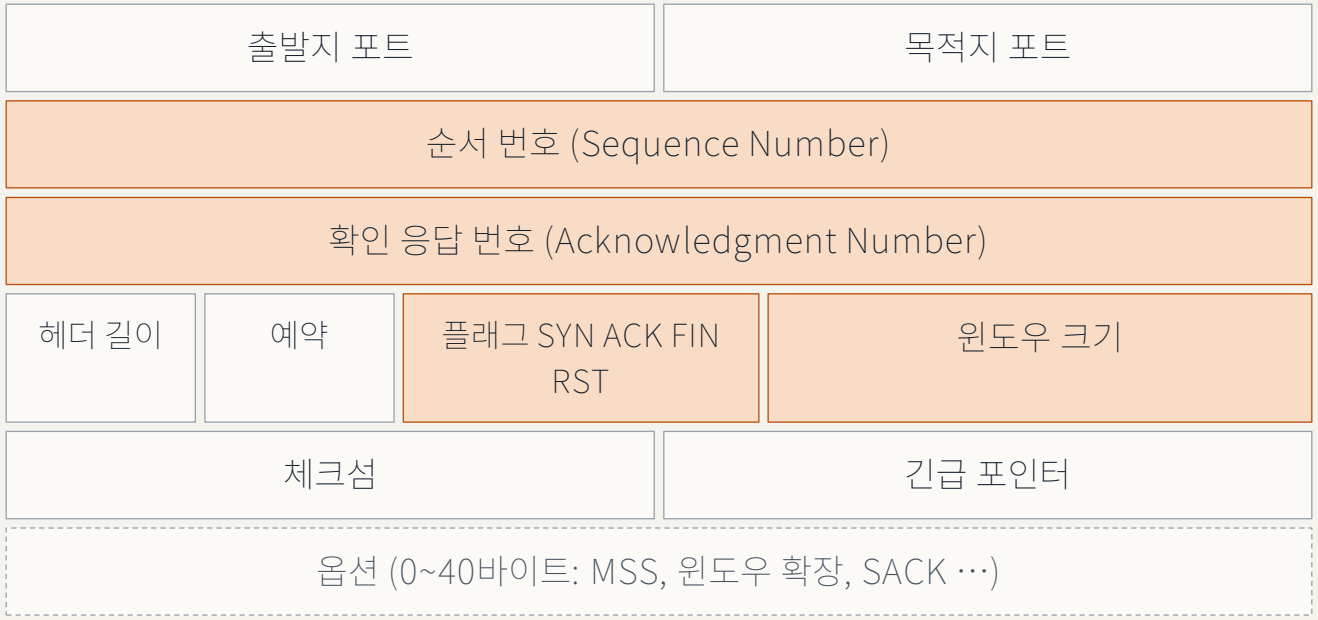
연결 하나는 (출발지 IP, 출발지 포트, 목적지 IP, 목적지 포트, 프로토콜) 다섯 개 값으로 구분합니다. 포트 포워딩에서 다시 나옵니다.

TCP와 UDP 한눈에 비교

구분	TCP	UDP
연결	연결 지향 (3-way handshake 후 전송)	비연결 (그냥 바로 보냄)
신뢰성	재전송 · 순서 보장 · 중복 제거	보장 안 함 (보내고 끝)
제어	흐름 제어 · 혼잡 제어	없음
데이터 경계	바이트 스트림 (경계 없음)	데이터그램 단위로 경계 유지
헤더 크기	20~60바이트	8바이트
통신 형태	1:1	1:1, 1:N (브로드캐스트 · 멀티캐스트)
대표 사용처	웹(HTTP/1.1 · 2), SSH, 이메일, DB 연결	DNS, 게임, 음성 · 영상 통화, DHCP, QUIC

헤더를 보면 차이가 보인다

TCP 헤더 · 20~60바이트



UDP 헤더 · 8바이트



끝. 정말 이게 전부입니다.

주황색 필드가 오늘의 주인공

- 순서 · 확인 번호 → 핸드셰이크, 재전송
- 플래그 → 연결 시작(SYN)과 종료(FIN)
- 윈도우 크기 → 흐름 제어

01 · TCP vs UDP

신뢰성이 없는 UDP, 왜 아직도 많이 쓸까?

① 늦게 온 데이터는 쓸모없다

게임 속 위치, 통화 음성은 1초 전 것을 재전송받아도 소용없음. 재전송을 기다리며 멈추느니 버리고 다음 것을 씀

② 한 번 묻고 끝나는 요청

DNS 조회는 질문 하나, 답 하나. 연결을 맺는 왕복 시간이 정작 보낼 데이터보다 비쌈

③ 여러 명에게 한 번에

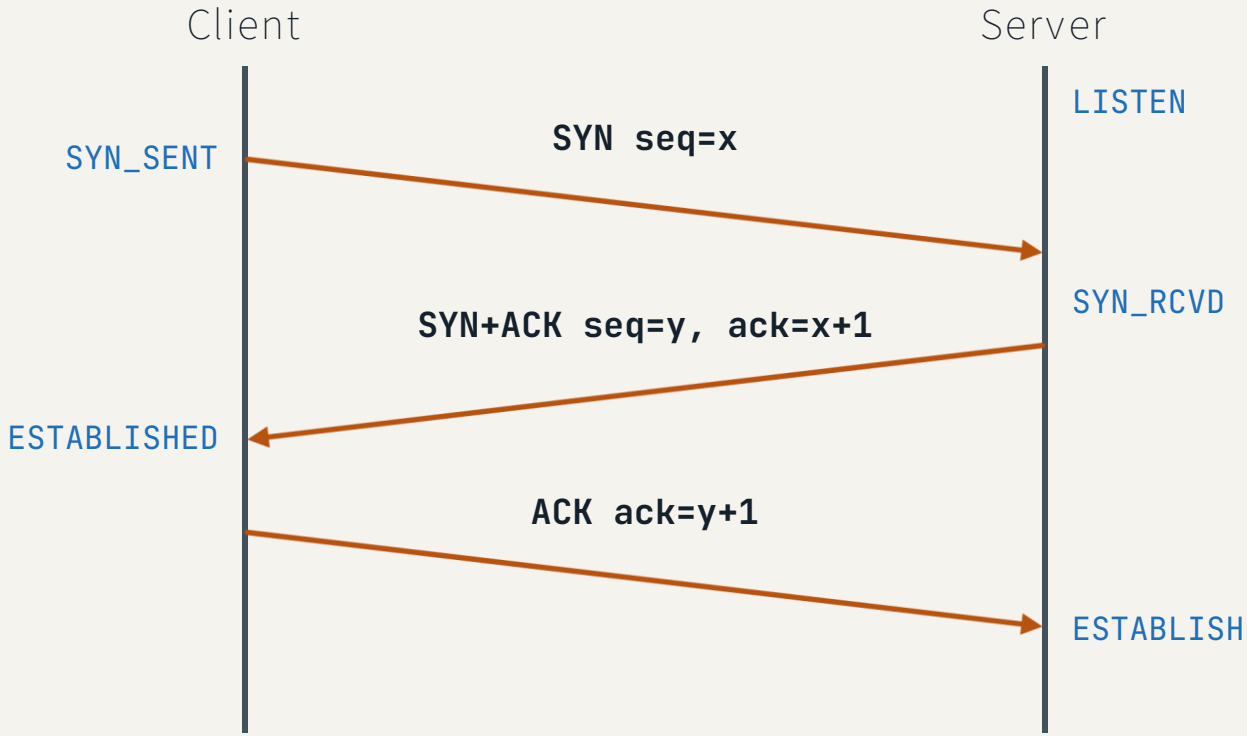
브로드캐스트 · 멀티캐스트는 '1:1 연결' 개념이 없어 UDP로만 가능. DHCP는 IP를 받기 전이라 브로드캐스트로 물어봄

④ 필요한 신뢰성만 직접 만든다

HTTP/3의 QUIC은 UDP 위에 재전송 · 혼잡 제어 · 암호화를 앱 쪽에서 구현. OS 안의 TCP는 고치기 어렵기 때문

UDP는 기능이 없는 게 아니라, 필요한 기능만 골라 올릴 수 있는 빈 도화지

3-way Handshake: 연결 맺기



seq와 ack

seq는 내가 보내는 데이터의 번호, ack는 '다음엔 이 번호를 줘'라는 뜻. 받은 번호 + 1

시작 번호는 무작위

x, y(ISN)를 0이 아닌 랜덤으로 정해 이전 연결의 패킷과 섞이거나 번호를 예측당하는 것을 막음

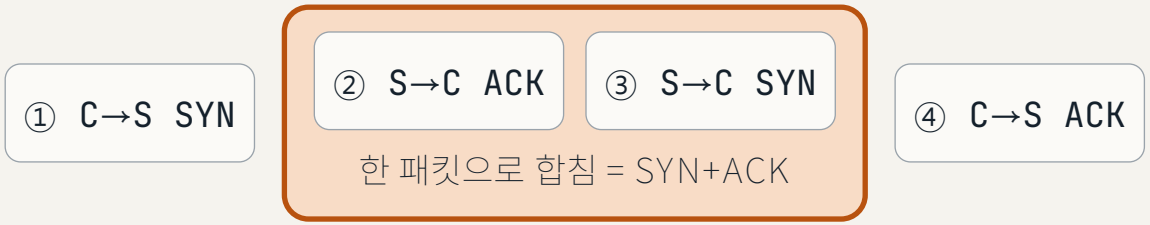
세 번째 ACK부터

양쪽 모두 ESTABLISHED. 세 번째 ACK에는 데이터를 실어 보낼 수도 있음

02 · TCP Handshake

왜 2번이 아니라 3번일까?

양쪽 모두 '내 번호를 보냈다'와 '네 번호를 받았다'를 확인해야 하니 필요한 메시지는 4개. 가운데 둘을 합치면 3개입니다.



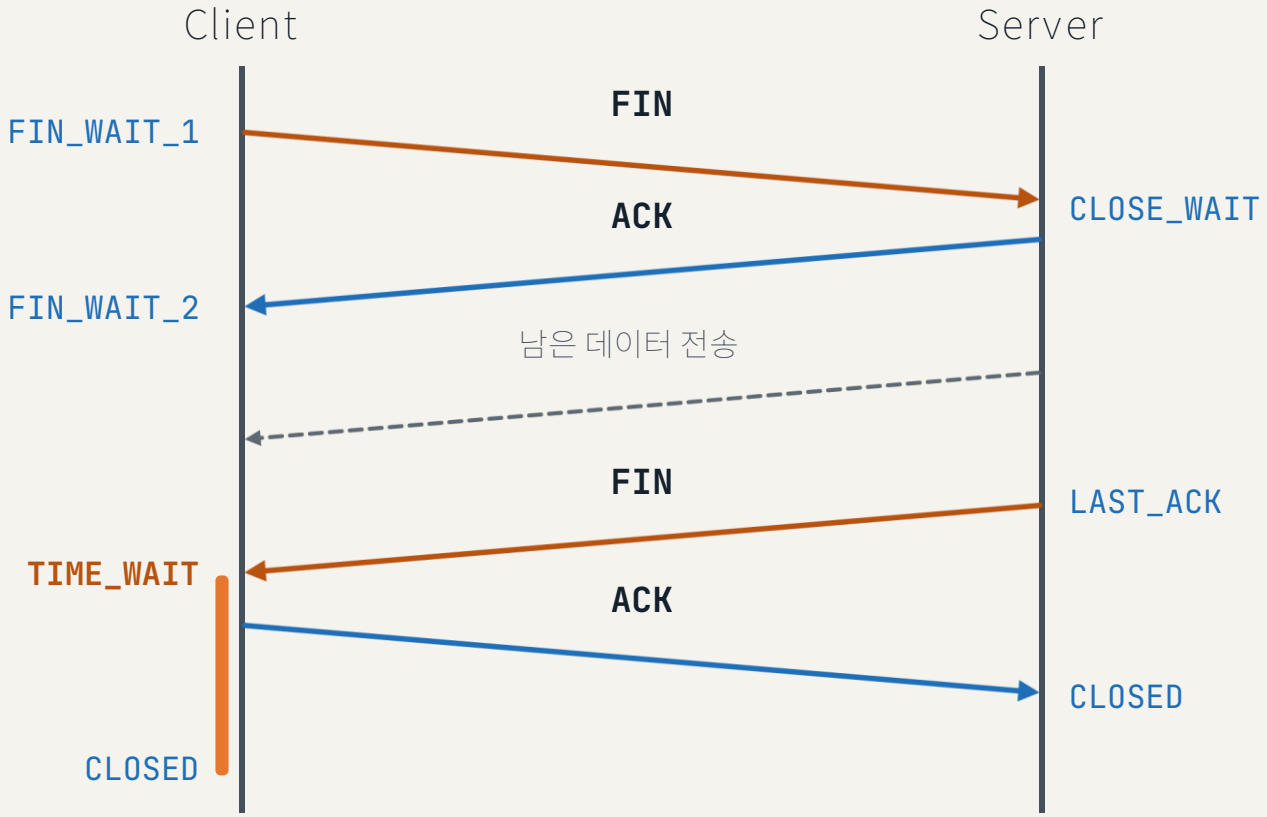
2번이면 ① 서버가 확신할 수 없다

SYN+ACK에서 끝나면, 서버는 자기가 보낸 번호 y가 클라이언트에 닿았는지 모른 채 데이터를 보내야 함

2번이면 ② 옛 SYN에 속는다

네트워크를 떠돌던 예전 SYN이 늦게 도착하면 서버가 연결을 열어 자원을 낭비. 3번이면 클라이언트가 RST로 거절

4-way Handshake: 연결 끊기



먼저 끊는 쪽은 누구든 OK

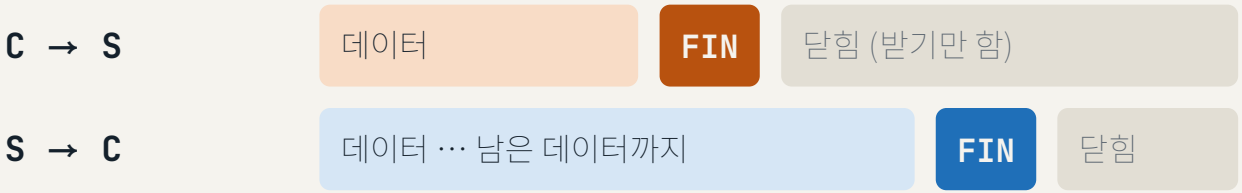
먼저 FIN을 보낸 쪽(Active Close)이 마지막에 TIME_WAIT를 겪음. 서버가 먼저 끊으면 서버가 겪음

CLOSE_WAIT

상대 FIN은 받았고, 내 프로그램이 close()를 부르길 기다리는 상태. 서버에 잔뜩 쌓여 있다면 코드에서 소켓을 안 닫은 버그

종료는 왜 4번일까?

TCP는 양방향이고, 두 방향을 각각 따로 닫습니다. FIN을 받은 쪽은 아직 보낼 데이터가 남아 있을 수 있어서 ACK와 FIN을 합치지 못합니다.



한쪽만 닫힌 이 상태를 Half-Close라고 합니다. 보낼 게 없으면 ACK와 FIN을 한 번에 보내 3번처럼 보이기도 합니다.

전화 끊기로 비유하면

“나 할 말 다 했어.” FIN

“응, 알았어. 근데 하나만 더…” ACK

“…이제 나도 끝!” FIN

“응, 끊을게.” ACK

TIME_WAIT: 먼저 끊은 쪽이 기다리는 이유

2MSL

MSL은 패킷이 네트워크에서 살아남는 최대 시간. RFC 기준 2분이고, 리눅스는 TIME_WAIT를 60초로 고정

① 마지막 ACK가 사라질 수 있다

ACK를 못 받은 상대는 FIN을 다시 보냄. 내가 이미 닫혀 있으면 RST만 돌아가 상대가 정상 종료를 못 함

② 늦게 온 옛 패킷이 섞일 수 있다

같은 IP · 포트 조합으로 새 연결을 바로 열면, 떠돌던 이전 연결의 패킷이 새 데이터로 오해받음

실무에서는 짧은 연결을 대량으로 맺고 끊는 쪽에 TIME_WAIT 소켓이 수만 개 쌓여 포트가 부족해짐 → Keep-Alive, 커넥션 풀로 연결을 재사용

세 가지 제어, 각자 지키는 대상이 다르다

지키는 대상 · 수신자

흐름 제어

받는 쪽 버퍼가 넘치지 않게 송신 속도를 맞춤

`rwnd · 슬라이딩 윈도우`

지키는 대상 · 네트워크

혼잡 제어

중간 라우터들이 넘치지 않게 전송량을 조절

`cwnd · Slow Start · AIMD`

지키는 대상 · 데이터

오류 제어

잃어버리거나 깨진 세그먼트를 찾아 다시 보냄

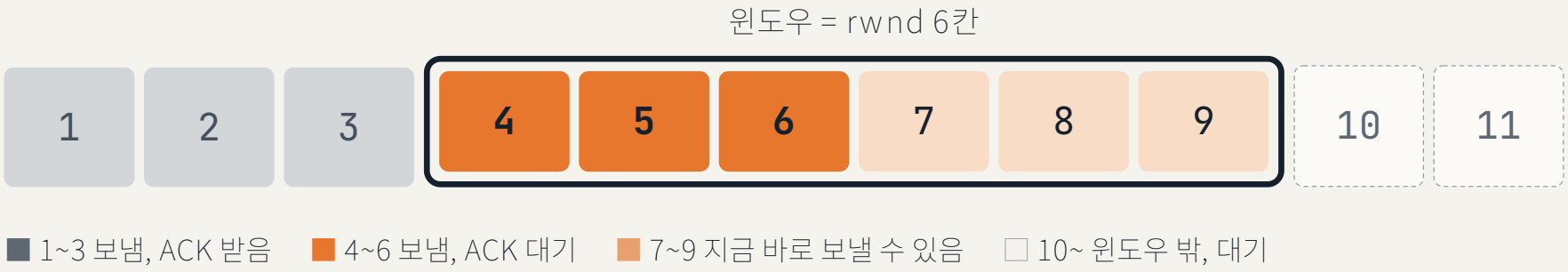
`체크섬 · ACK · 재전송`

그래서 송신자가 한 번에 보낼 수 있는 양은

min(rwnd, cwnd)

흐름 제어: 슬라이딩 윈도우

Stop-and-Wait는 하나 보내고 ACK가 올 때까지 대기 → 왕복 시간 내내 논다. 슬라이딩 윈도우는 ACK 없이도 윈도우 크기만큼 연달아 보낸다.



- 수신자는 ACK마다 헤더의 '윈도우 크기'에 남은 버퍼 공간(rwnd)을 실어 보냄
- ACK 4가 오면 윈도우가 한 칸 오른쪽으로 미끄러져 10을 보낼 수 있게 됨
- rwnd = 0이면 전송 중단, 가끔 Zero Window Probe로 '이제 받을 수 있어?' 확인

혼잡 제어: Slow Start와 AIMD

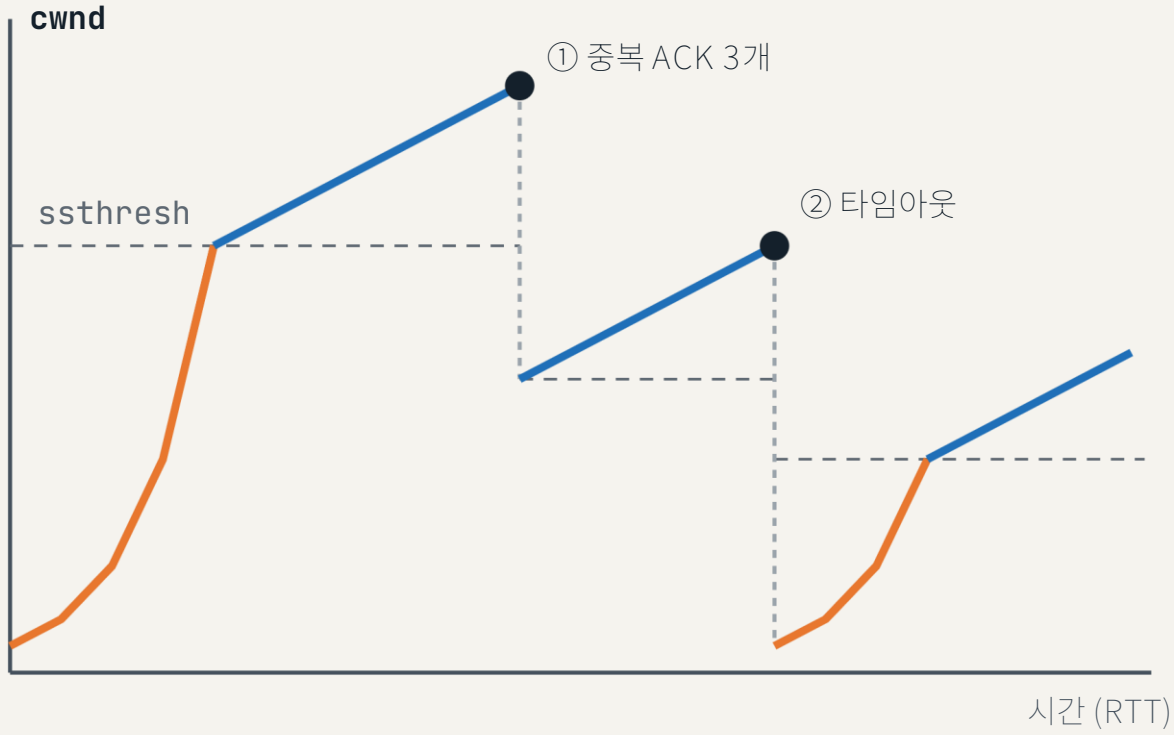
cwnd는 송신자가 '지금 네트워크가 이만큼은 버틴다'고 추측해 정한 한도. 패킷 손실을 혼잡 신호로 봅니다.

Slow Start

1 MSS에서 시작해 ACK 하나마다 +1 → RTT마다 2배. 이름과 달리 지수적으로 빠르게 ssthresh까지 올라감

혼잡 회피 = AIMD

ssthresh를 넘으면 RTT마다 +1(Additive Increase), 손실이 나면 절반으로(Multiplicative Decrease) → 톱니 모양



손실을 감지하면 얼마나 줄일까?

손실 신호	무슨 뜻일까	대응 (TCP Reno)
① 중복 ACK 3개	뒤 패킷들은 도착함 → 일부만 빠졌고 네트워크는 아직 살아 있음	cwnd를 절반으로 → 그 자리에서 혼잡 회피 계속 (빠른 재전송 · 빠른 회복)
② 타임아웃	ACK가 아예 안 옴 → 심각한 혼잡	ssthresh는 절반, cwnd는 1 → Slow Start부터 다시

면접 답변: Slow Start vs AIMD

Slow Start는 처음이나 리셋 직후 적정선을 빠르게 찾는 단계(지수 증가), AIMD는 적정선 근처를 조심스럽게 탐색하는 단계(선형 증가 · 절반 감소). 둘은 경쟁 관계가 아니라 한 알고리즘 안의 두 단계입니다.

요즘은?

Tahoe는 둘 다 1로 리셋하던 초기 방식. 리눅스 기본은 CUBIC, 구글의 BBR은 손실 대신 대역폭과 RTT를 측정

오류 제어: 잃어버리면 다시 보낸다

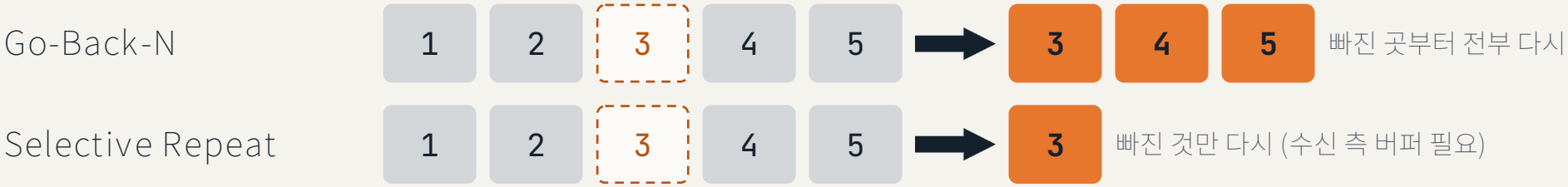
검출
체크섬이 틀리면 조용히 버림

→

확인
ACK = 다음에 받을 번호

→

재전송
타임아웃 또는 중복 ACK 3개

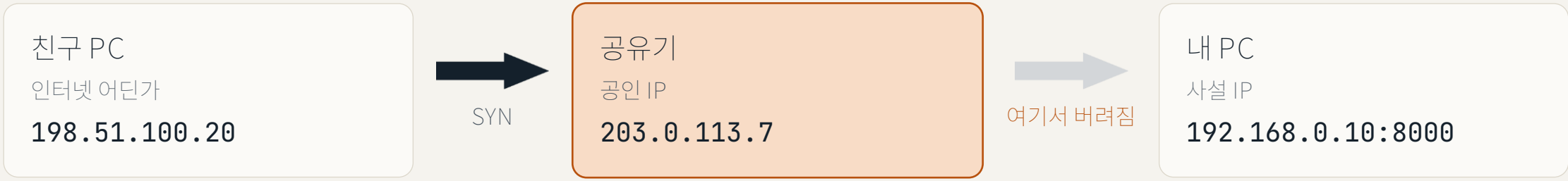


점선 3 = 전송 중 유실된 세그먼트

실제 TCP는? 기본은 누적 ACK(Go-Back-N에 가까움)이고, SACK 옵션으로 '4, 5는 받았어'를 알려 빠진 3만 재전송(Selective Repeat에 가까움)

04 · 포트 포워딩

집에서 띄운 서버, 왜 친구는 못 들어올까?



① 사설 IP는 인터넷에 길이 없다

10.x · 172.16~31.x · 192.168.x(RFC 1918)는 집집마다 겹치는 주소라 인터넷 라우터가 어디로 보낼지 모름

② 공유기는 모르는 손님을 막는다

공유기는 안에서 먼저 나간 연결만 기억함. 밖에서 먼저 온 SYN은 변환 테이블에 짝이 없어 버려짐

NAPT: 공인 IP 하나를 여럿이 나눠 쓰기

내부 기기 (사설)	공유기가 바꾼 주소 (공인)	목적지	프로토콜
192.168.0.10:51000	203.0.113.7:40001	198.51.100.20:443	TCP
192.168.0.11:51000	203.0.113.7:40002	198.51.100.20:443	TCP
192.168.0.12:52344	203.0.113.7:40003	198.51.100.53:53	UDP

나갈 때

출발지를 공인 IP:새 포트로 바꾸고 테이블에 기록. 두 기기가 똑같이 51000을 써도 바꾼 포트로 구분됨

돌아올 때

응답의 목적지 포트(예: 40002)로 테이블을 찾아 192.168.0.11:51000으로 되돌려 전달

NAT는 IP만 1:1로 바꾸고, NAPT(PAT)는 포트까지 바꿔 N:1로 공유합니다. 집 공유기가 하는 일은 사실 NAPT.

포트 포워딩: 밖에서 들어오는 길을 미리 열기

공유기에 넣는 규칙 **203.0.113.7:8080/TCP → 192.168.0.10:8000**

들어올 때
DNAT · 목적지 변경

출발 198.51.100.20:53211 목적 203.0.113.7:8080	→ 출발 198.51.100.20:53211 **목적 192.168.0.10:8000**

나갈 때 (응답)
출발지를 되돌림

출발 192.168.0.10:8000 목적 198.51.100.20:53211	→ **출발 203.0.113.7:8080** 목적 198.51.100.20:53211

응답의 출발지를 공인 주소로 바꾸는 건 흔히 SNAT로 설명합니다. 정확히는 들어올 때 남긴 DNAT 기록(연결 추적)을 거꾸로 적용하는 것. 친구는 처음부터 끝까지 203.0.113.7:8080과 대화한다고 느낍니다.

NAPT와 포트 포워딩 정리

구분	NAPT	포트 포워딩
누가 먼저 연결?	안 → 밖 (내부 기기가 시작)	밖 → 안 (외부에서 시작)
규칙	자동 · 연결마다 생기고 만료	사람이 미리 넣는 고정 규칙
바꾸는 주소	나갈 때 출발지 (SNAT)	들어올 때 목적지 (DNAT)
쓰임	평소 인터넷 사용 전부	집 서버, 게임 서버, NAS

포트는 전송 계층의 주소

그래서 규칙에 TCP/UDP를 고름. 공유기는 IP 헤더뿐 아니라 TCP · UDP 헤더의 포트까지 고치고 체크섬도 다시 계산

보안 주의

포워딩은 방화벽에 구멍을 내는 일. 필요한 포트만 열고, 다 쓰면 닫기. DB 포트(3306 등)는 열지 않기

05 · 생각해볼 질문 ①

ngrok은 공유기 설정 없이 어떻게 공개할까?

밖 → 안이 막혀 있다면, 안 → 밖으로 먼저 연결해 두면 된다



1. 내 PC의 ngrok agent가 ngrok 서버로 나가는 연결을 먼저 맺음 → 공유기엔 평범한 NAPT 항목
2. ngrok 서버가 공인 주소(https://….ngrok-free.app)를 발급
3. 외부 요청은 ngrok 서버로 가고, 이미 열린 터널을 거꾸로 타고 agent까지 내려옴
4. agent가 localhost:8000에 넘기고, 응답은 같은 길로 되돌아감

같은 원리: ssh -R(리버스 터널), Cloudflare Tunnel · 대가: 모든 트래픽이 중계 서버를 거쳐 지연이 늘고, 무료 플랜 제약이 있음

05 · 생각해볼 질문 ②

포트가 고갈되면 어떻게 될까?

65,535

포트 번호는 16비트. 클라이언트가 쓰는 임시 포트는 리눅스 기본 32768~60999, 약 2.8만 개뿐

① 공유기 · 통신사 NAT에서

공인 IP 하나의 포트를 많은 사용자가 나눠 씀 → 테이블이 꽉 차면 새 연결의 SYN이 버려져 접속 타임아웃

② 내 서버가 클라이언트일 때

같은 목적지로 짧은 연결을 반복하면 TIME_WAIT 60초 동안 포트가 묶임 → 초당 약 470개를 넘으면 고갈, connect() 실패

해결 방법

- Keep-Alive · 커넥션 풀로 연결 재사용
- 공인 IP 여러 개를 풀로 묶기 (CGNAT)
- 임시 포트 범위 확대 · tcp_tw_reuse
- IPv6로 NAT 자체를 없애기

정리

면접 예상 질문, 한 줄 답변

Q. TCP와 UDP의 차이는?

TCP는 연결 · 재전송 · 순서 보장 · 흐름/혼잡 제어로 신뢰성을, UDP는 비연결 · 8바이트 헤더로 속도와 단순함을 택함

Q. UDP를 아직도 쓰는 이유는?

늦은 데이터가 무의미한 실시간 서비스, 짧은 요청(DNS), 1:N 전송, 그리고 QUIC처럼 신뢰성을 직접 만들기 위해

Q. 연결은 왜 3-way인가요?

양방향 각각 시작 번호를 보내고 확인해야 하는데, 서버의 ACK와 SYN을 합칠 수 있어 최소 3번

Q. 종료는 왜 4-way인가요?

두 방향을 따로 닫기 때문. FIN을 받은 쪽은 남은 데이터를 다 보낸 뒤 FIN을 보내므로 ACK와 FIN이 분리됨

Q. TIME_WAIT는 왜 필요한가요?

마지막 ACK 유실 시 재전송된 FIN에 답하고, 이전 연결의 지연 패킷이 새 연결에 섞이지 않게 2MSL 대기

Q. 포트 포워딩은 왜 필요한가요?

사설 IP는 외부에서 라우팅되지 않고 NAPT는 안에서 시작한 연결만 기억하므로, 밖에서 들어올 고정 규칙(DNAT)이 필요

참고 자료

더 읽어볼 곳

- RFC 9293 · TCP (RFC 793 대체)
- RFC 768 · UDP
- RFC 5681 · TCP 혼잡 제어
- RFC 2018 · TCP SACK
- RFC 9000 · QUIC
- RFC 1918 · 사설 IP 대역
- RFC 3022 · NAT와 NAPT
- RFC 6598 · CGNAT 공유 주소 대역
- ngrok 공식 문서
- gyoogle 기술 면접 정리